

Review of *Lec11 Processor Arch: Sequential*

吴悠¹

¹School of Physics, Peking University

✉ 2300011348@stu.pku.edu.cn

Oct 22, 2025

Contents

1. Y86-64 and building blocks
2. SEQ stages
3. SEQ hardware structure
4. SEQ timing
5. Surveying SEQ

PART 01

Quick review of ISA & logic design

- *Y86-64 instructions*
- *Codes and identifiers*

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
rrmovq rA , rB	2	0	rA	rB						
irmovq V , rB	3	0	F	rB					V	
rmmovq rA , D(rB)	4	0	rA	rB					D	
mrmovq D(rB) , rA	5	0	rA	rB					D	
OPq rA , rB	6	fn	rA	rB						
jXX Dest	7	fn							Dest	
cmovXX rA , rB	2	fn	rA	rB						
call Dest	8	0							Dest	
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

Figure 4.2 Y86-64 instruction set. Instruction encodings range between 1 and 10 bytes. An instruction consists of a 1-byte instruction specifier, possibly a 1-byte register specifier, and possibly an 8-byte constant word. Field **fn** specifies a particular integer operation (OPq), data movement condition (cmovXX), or branch condition (jXX). All numeric values are shown in hexadecimal.



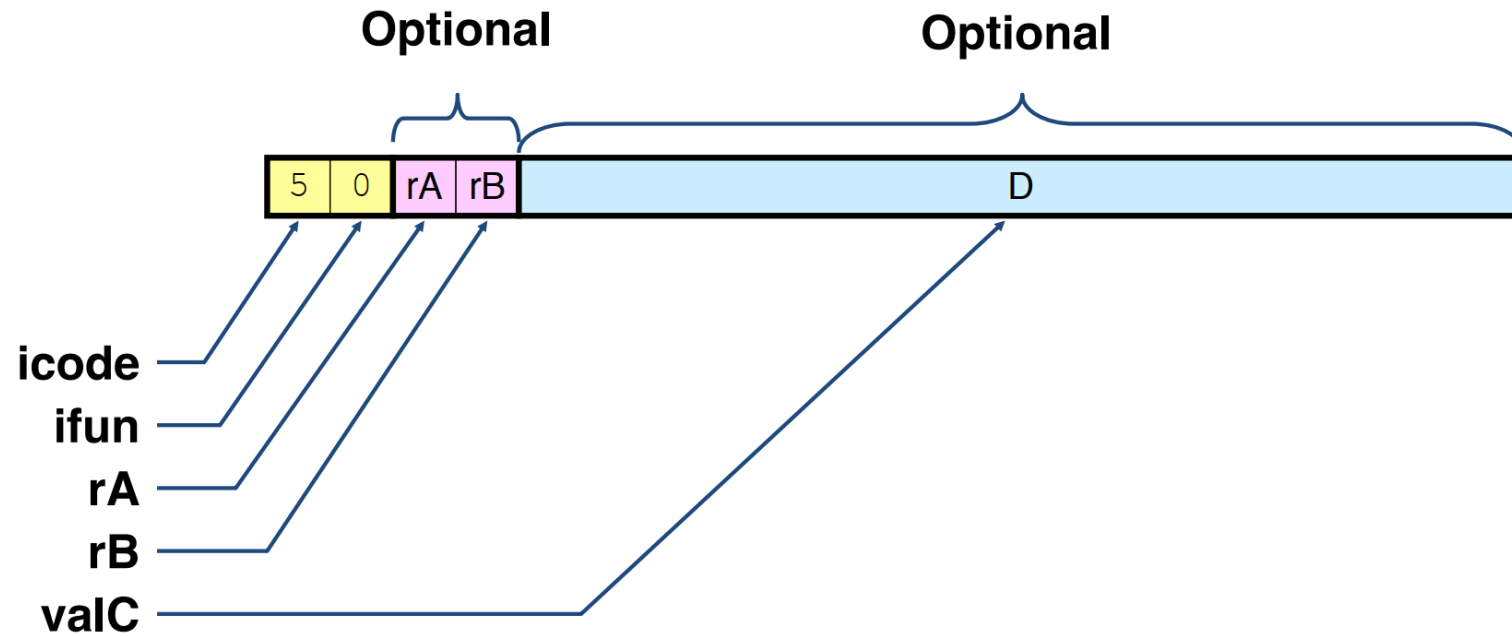
Operations	Branches		Moves											
addq <table><tr><td>6</td><td>0</td></tr></table>	6	0	jmp <table><tr><td>7</td><td>0</td></tr></table>	7	0	jne <table><tr><td>7</td><td>4</td></tr></table>	7	4	rrmovq <table><tr><td>2</td><td>0</td></tr></table>	2	0	cmovne <table><tr><td>2</td><td>4</td></tr></table>	2	4
6	0													
7	0													
7	4													
2	0													
2	4													
subq <table><tr><td>6</td><td>1</td></tr></table>	6	1	jle <table><tr><td>7</td><td>1</td></tr></table>	7	1	jge <table><tr><td>7</td><td>5</td></tr></table>	7	5	cmovle <table><tr><td>2</td><td>1</td></tr></table>	2	1	cmovge <table><tr><td>2</td><td>5</td></tr></table>	2	5
6	1													
7	1													
7	5													
2	1													
2	5													
andq <table><tr><td>6</td><td>2</td></tr></table>	6	2	jnl <table><tr><td>7</td><td>2</td></tr></table>	7	2	jg <table><tr><td>7</td><td>6</td></tr></table>	7	6	cmovl <table><tr><td>2</td><td>2</td></tr></table>	2	2	cmovg <table><tr><td>2</td><td>6</td></tr></table>	2	6
6	2													
7	2													
7	6													
2	2													
2	6													
xorq <table><tr><td>6</td><td>3</td></tr></table>	6	3	je <table><tr><td>7</td><td>3</td></tr></table>	7	3		cmove <table><tr><td>2</td><td>3</td></tr></table>	2	3					
6	3													
7	3													
2	3													

Figure 4.3 Function codes for Y86-64 instruction set. The code specifies a particular integer operation, branch condition, or data transfer condition. These instructions are shown as OPq, jXX, and cmovXX in Figure 4.2.

Number	Register name	Number	Register name
0	%rax	8	%r8
1	%rcx	9	%r9
2	%rdx	A	%r10
3	%rbx	B	%r11
4	%rsp	C	%r12
5	%rbp	D	%r13
6	%rsi	E	%r14
7	%rdi	F	No register

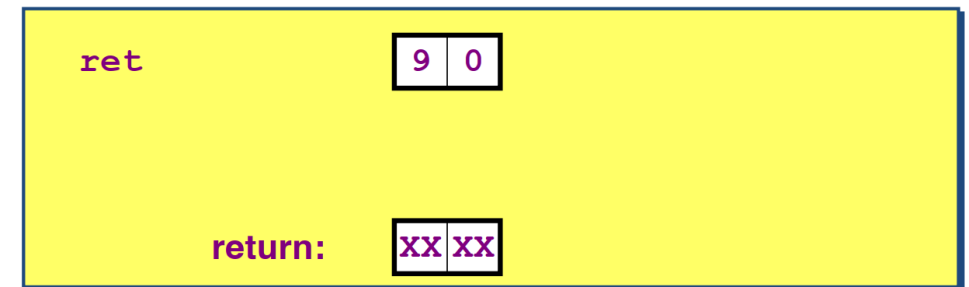
Figure 4.4 Y86-64 program register identifiers. Each of the 15 program registers has an associated identifier (ID) ranging from 0 to 0xE. ID 0xF in a register field of an instruction indicates the absence of a register operand.

Y86-64 instruction encoding



■ Instruction Format

- Instruction byte icode:ifun
- Optional register byte rA:rB
- Optional constant word valC



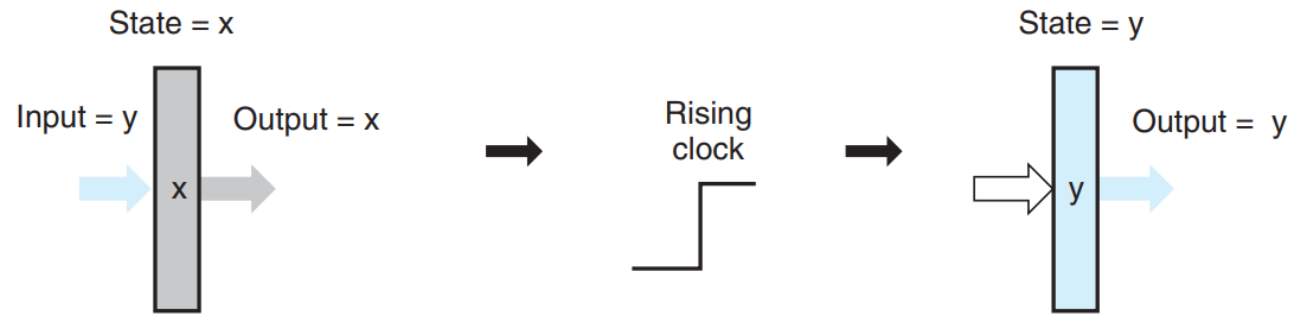
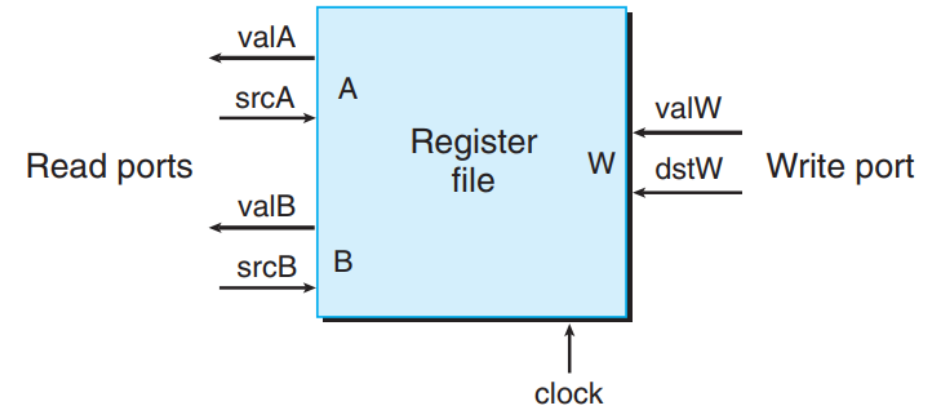


Figure 4.16 Register operation. The register outputs remain held at the current register state until the clock signal rises. When the clock rises, the values at the register inputs are captured to become the new register state.

The following diagram shows a typical register file:



Our processor has a random access memory for storing program data, as illustrated below:

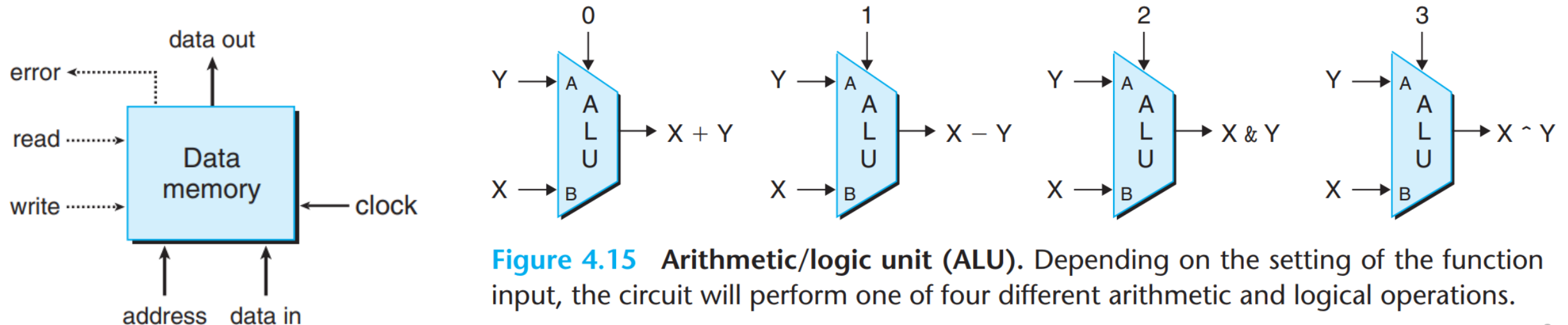


Figure 4.15 Arithmetic/logic unit (ALU). Depending on the setting of the function input, the circuit will perform one of four different arithmetic and logical operations.



PART 02

SEQ Stages

Fetch

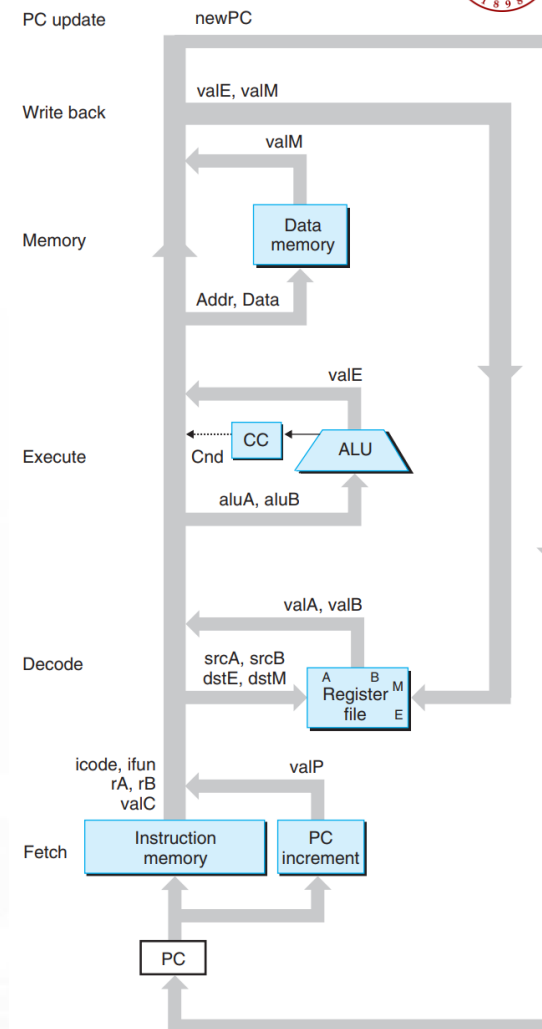
Decode

Execute

Memory

Write back

PC update





- 1. On each clock cycle, SEQ performs all the steps required to process a complete instruction.*
- 2. The processor loops indefinitely, performing these stages.*
- 3. The processor will stop when any exception occurs—that is, when it executes a halt or invalid instruction, or it attempts to read or write an invalid address.*

Fetch. The fetch stage reads the bytes of an instruction from memory, using the program counter (PC) as the memory address. From the instruction it extracts the two 4-bit portions of the instruction specifier byte, referred to as icode (the instruction code) and ifun (the instruction function). It possibly fetches a register specifier byte, giving one or both of the register operand specifiers rA and rB. It also possibly fetches an 8-byte constant word valC. It computes valP to be the address of the instruction following the current one in sequential order. That is, valP equals the value of the PC plus the length of the fetched instruction.

`rmmovq rA, D(rB)`

4	0	rA	rB	D			
---	---	----	----	---	--	--	--

■ Fetch

- Read 10 bytes

■ Memory

- Write to memory

Stage computation



Stage	OPq rA, rB	rrmovq rA, rB	irmovq V, rB
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 2$	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 2$	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$ $\text{valC} \leftarrow M_8[\text{PC} + 2]$ $\text{valP} \leftarrow \text{PC} + 10$
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow R[\text{rB}]$	$\text{valA} \leftarrow R[\text{rA}]$	
Execute	$\text{valE} \leftarrow \text{valB OP valA}$ Set CC	$\text{valE} \leftarrow 0 + \text{valA}$	$\text{valE} \leftarrow 0 + \text{valC}$
Memory			
Write back	$R[\text{rB}] \leftarrow \text{valE}$	$R[\text{rB}] \leftarrow \text{valE}$	$R[\text{rB}] \leftarrow \text{valE}$
PC update	$\text{PC} \leftarrow \text{valP}$	$\text{PC} \leftarrow \text{valP}$	$\text{PC} \leftarrow \text{valP}$

Figure 4.18 Computations in sequential implementation of Y86-64 instructions OPq, rrmovq, and irmovq. These instructions compute a value and store the result in a register. The notation icode:ifun indicates the two components of the instruction byte, while rA:rB indicates the two components of the register specifier byte. The notation $M_1[x]$ indicates accessing (either reading or writing) 1 byte at memory location x , while $M_8[x]$ indicates accessing 8 bytes.

Stage computation

Stage	Computation	OPq rA, rB	mrmovq D(rB), rA
Fetch	icode, ifun rA, rB valC valP	icode:ifun $\leftarrow M_1[PC]$ rA:rB $\leftarrow M_1[PC + 1]$ valP $\leftarrow PC + 2$	icode:ifun $\leftarrow M_1[PC]$ rA:rB $\leftarrow M_1[PC + 1]$ valC $\leftarrow M_8[PC + 2]$ valP $\leftarrow PC + 10$
Decode	valA, srcA valB, srcB	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valB $\leftarrow R[rB]$
Execute	valE Cond. codes	valE $\leftarrow$ valB OP valA Set CC	valE $\leftarrow$ valB + valC
Memory	Read/write		valM $\leftarrow M_8[valE]$
Write back	E port, dstE M port, dstM	R[rB] $\leftarrow$ valE	R[rA] $\leftarrow$ valM
PC update	PC	PC $\leftarrow$ valP	PC $\leftarrow$ valP

Figure 4.24 Identifying the different computation steps in the sequential implementation. The second column identifies the value being computed or the operation being performed in the stages of SEQ. The computations for instructions OPq and mrmovq are shown as examples of the computations.

Stage computation

Stage	jXX Dest	call Dest	ret
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 9$	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 9$	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{valP} \leftarrow \text{PC} + 1$
Decode		$\text{valB} \leftarrow R[\%rsp]$	$\text{valA} \leftarrow R[\%rsp]$ $\text{valB} \leftarrow R[\%rsp]$
Execute	$\text{Cnd} \leftarrow \text{Cond}(\text{CC}, \text{ifun})$	$\text{valE} \leftarrow \text{valB} + (-8)$	$\text{valE} \leftarrow \text{valB} + 8$
Memory		$M_8[\text{valE}] \leftarrow \text{valP}$	$\text{valM} \leftarrow M_8[\text{valA}]$
Write back		$R[\%rsp] \leftarrow \text{valE}$	$R[\%rsp] \leftarrow \text{valE}$
PC update	$\text{PC} \leftarrow \text{Cnd} ? \text{valC} : \text{valP}$	$\text{PC} \leftarrow \text{valC}$	$\text{PC} \leftarrow \text{valM}$

Figure 4.21 Computations in sequential implementation of Y86-64 instructions jXX, call, and ret. These instructions cause control transfers.

Principle: No reading back



The processor never needs to read back the state updated by an instruction in order to complete the processing of this instruction. ■

Stage	pushq rA	popq rA
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$
	$\text{valP} \leftarrow \text{PC} + 2$	$\text{valP} \leftarrow \text{PC} + 2$
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow R[\%rsp]$	$\text{valA} \leftarrow R[\%rsp]$ $\text{valB} \leftarrow R[\%rsp]$
Execute	$\text{valE} \leftarrow \text{valB} + (-8)$	$\text{valE} \leftarrow \text{valB} + 8$
Memory	$M_8[\text{valE}] \leftarrow \text{valA}$	$\text{valM} \leftarrow M_8[\text{valA}]$
Write back	$R[\%rsp] \leftarrow \text{valE}$	$R[\%rsp] \leftarrow \text{valE}$ $R[\text{rA}] \leftarrow \text{valM}$
PC update	$\text{PC} \leftarrow \text{valP}$	$\text{PC} \leftarrow \text{valP}$

Figure 4.20 Computations in sequential implementation of Y86-64 instructions pushq and popq. These instructions push and pop the stack.

PART 03

SEQ hardware structure

Fetch

Decode

Execute

Memory

Write back

PC update



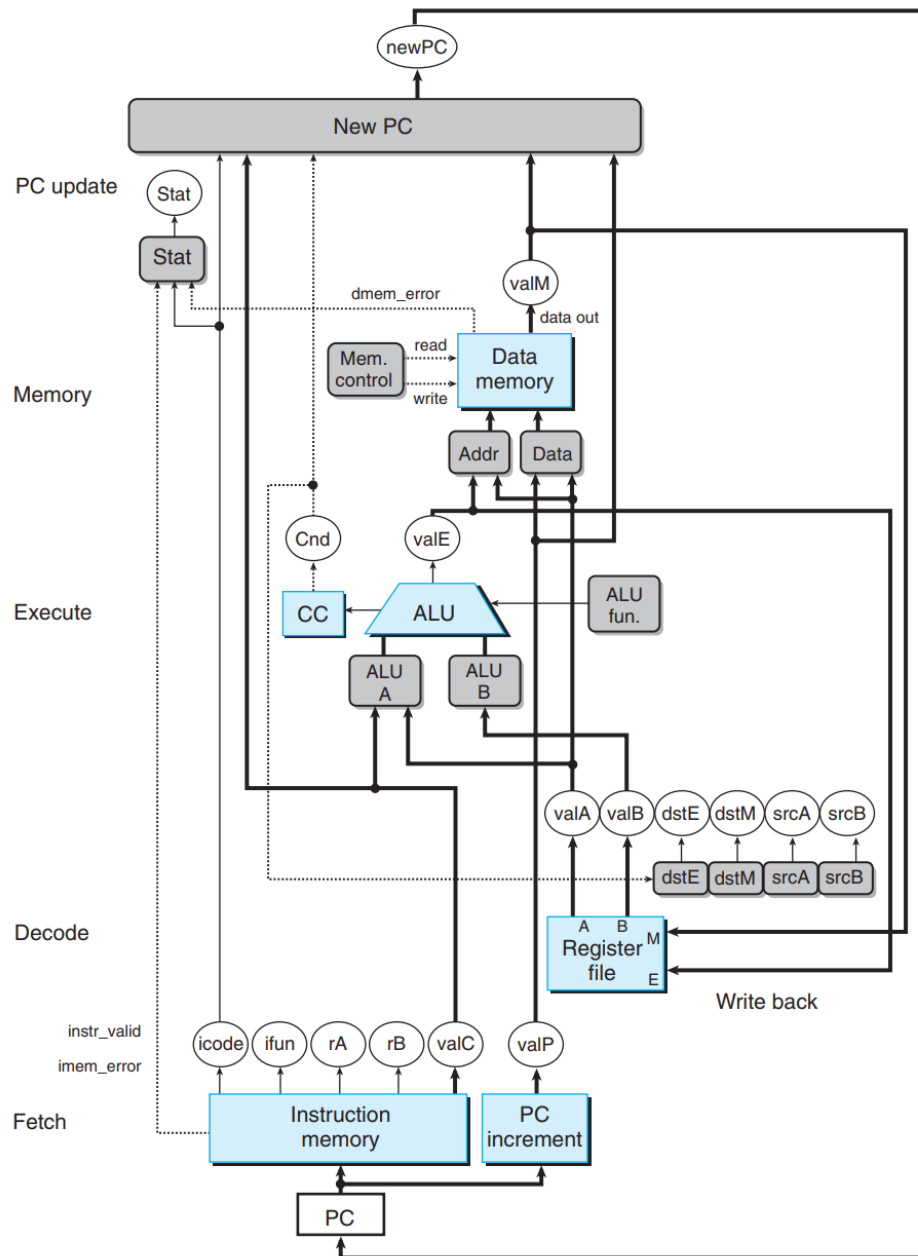
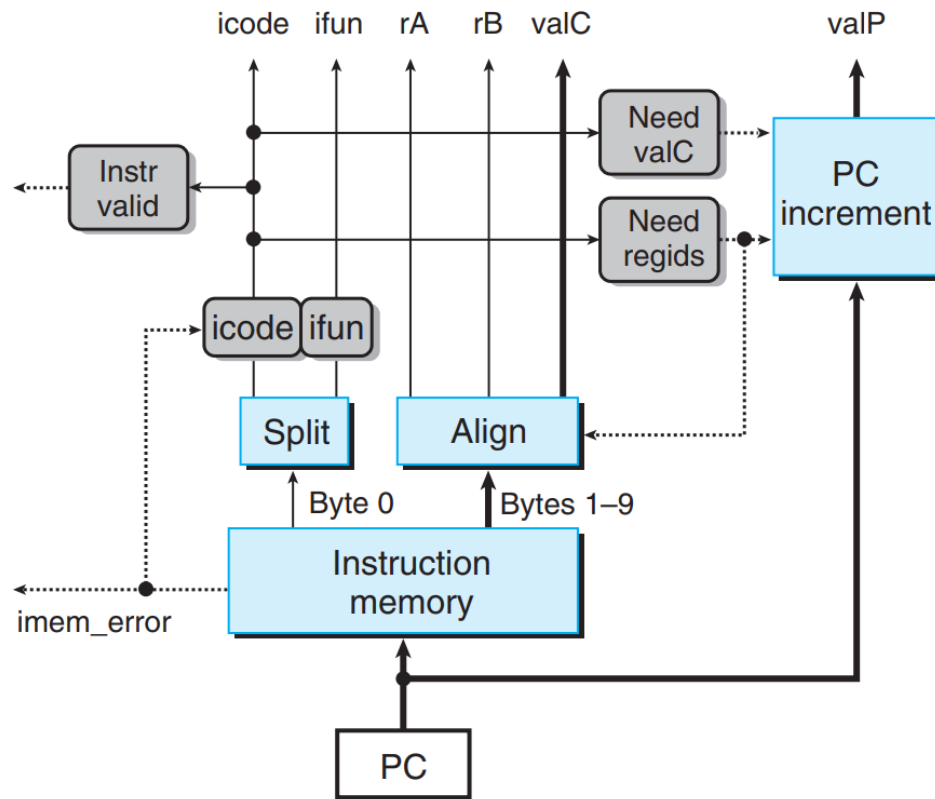


Figure 4.23 Hardware structure of SEQ, a sequential implementation. Some of the control signals, as well as the register and control word connections, are not shown.

Figure 4.23 gives a more detailed view of the hardware required to implement SEQ (although we will not see the complete details until we examine the individual stages). We see the same set of hardware units as earlier, but now the wires are shown explicitly. In this figure, as well as in our other hardware diagrams, we use the following drawing conventions:

- *Clocked registers are shown as white rectangles.* The program counter PC is the only clocked register in SEQ.
- *Hardware units are shown as light blue boxes.* These include the memories, the ALU, and so forth. We will use the same basic set of units for all of our processor implementations. We will treat these units as “black boxes” and not go into their detailed designs.
- *Control logic blocks are drawn as gray rounded rectangles.* These blocks serve to select from among a set of signal sources or to compute some Boolean function. We will examine these blocks in complete detail, including developing HCL descriptions.
- *Wire names are indicated in white circles.* These are simply labels on the wires, not any kind of hardware element.
- *Word-wide data connections are shown as medium lines.* Each of these lines actually represents a bundle of 64 wires, connected in parallel, for transferring a word from one part of the hardware to another.
- *Byte and narrower data connections are shown as thin lines.* Each of these lines actually represents a bundle of four or eight wires, depending on what type of values must be carried on the wires.
- *Single-bit connections are shown as dotted lines.* These represent control values passed between the units and blocks on the chip.

SEQ fetch stage



This unit reads 10 bytes from memory at a time, using the PC as the address of the first byte (byte 0). This byte is interpreted as the instruction byte and is split (by the unit labeled “Split”) into two 4-bit quantities.

Based on the value of **icode**, we can compute three 1-bit signals
instr_valid, **need_regids** and **need_valC**

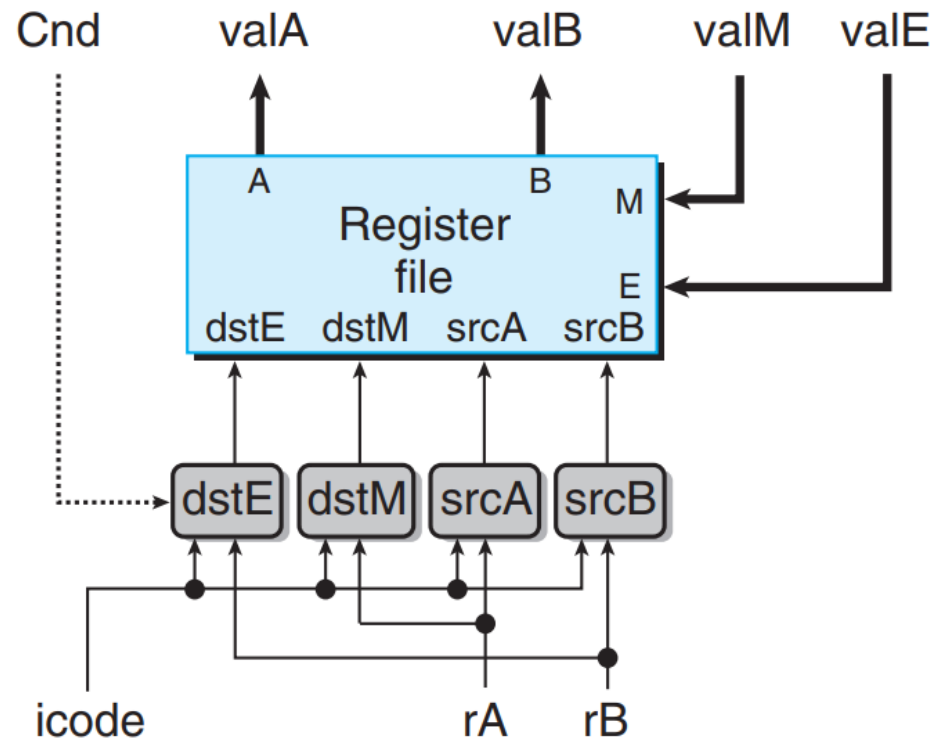
```
bool need_regids =  
  icode in {IRRMVQ, IOPQ, IPUSHQ,  
            IPOQ, IIRMOVQ, IRMMOVQ, IMRMVQ };
```


Decode and Write-Back Stages

```
int srcA = [  
    icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ } : rA;  
    icode in { IPOPQ, IRET } : RRSP;  
    1 : RNONE; # Don't need register  
];
```

Figure 4.28

SEQ decode and write-back stage. The instruction fields are decoded to generate register identifiers for four addresses (two read and two write) used by the register file. The values read from the register file become the signals *valA* and *valB*. The two write-back values *valE* and *valM* serve as the data for the writes.

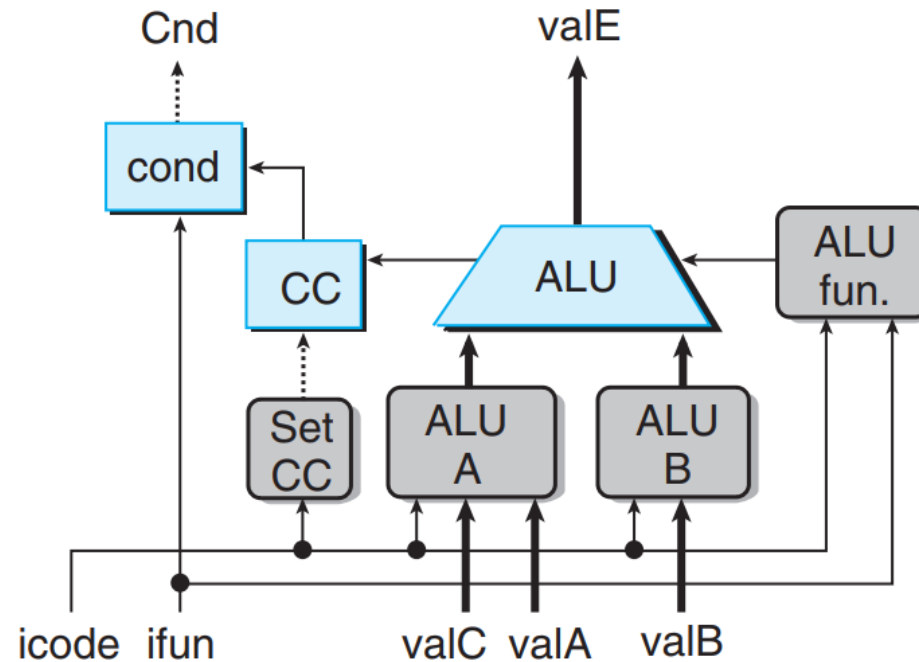


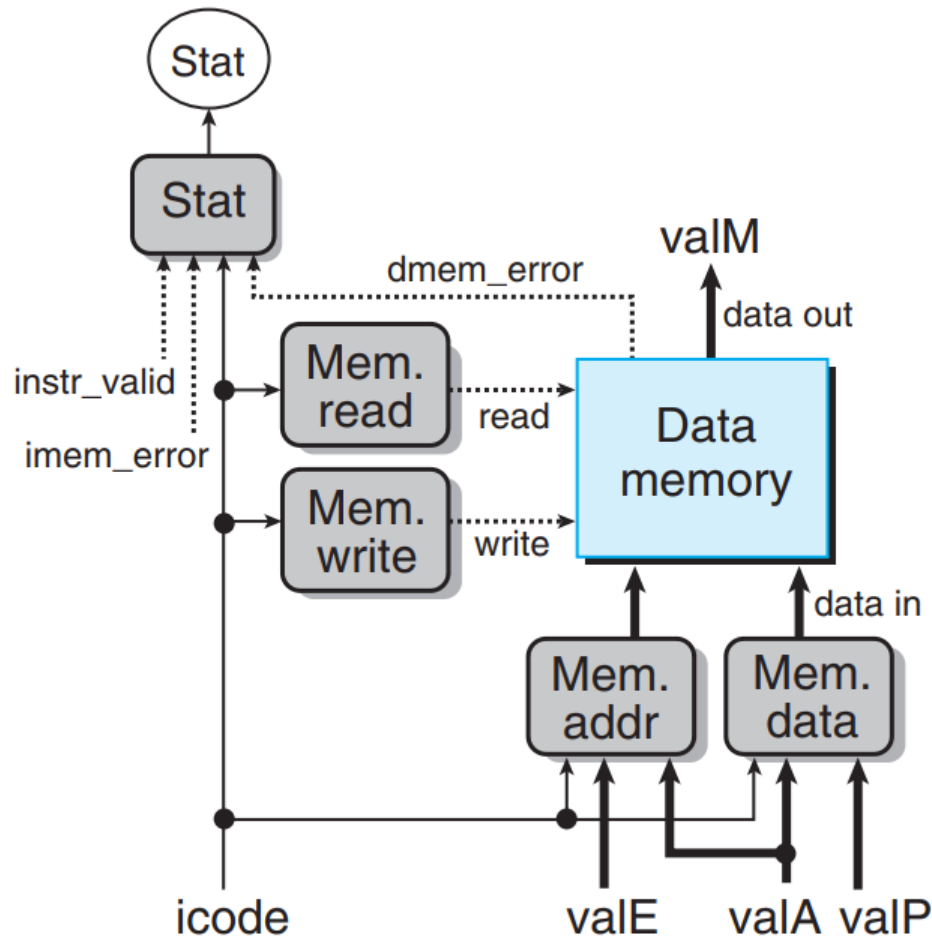
Execute stage

```
int aluA = [  
    icode in { IRRMOVQ, IOPQ } : valA;  
    icode in { IIRMOVQ, IRMMOVQ, IMRMVQ } : valC;  
    icode in { ICALL, IPUSHQ } : -8;  
    icode in { IRET, IPOPOPQ } : 8; # Other instructions don't need ALU  
];
```

Figure 4.29

SEQ execute stage. The ALU either performs the operation for an integer operation instruction or acts as an adder. The condition code registers are set according to the ALU value. The condition code values are tested to determine whether a branch should be taken.





Control Logic

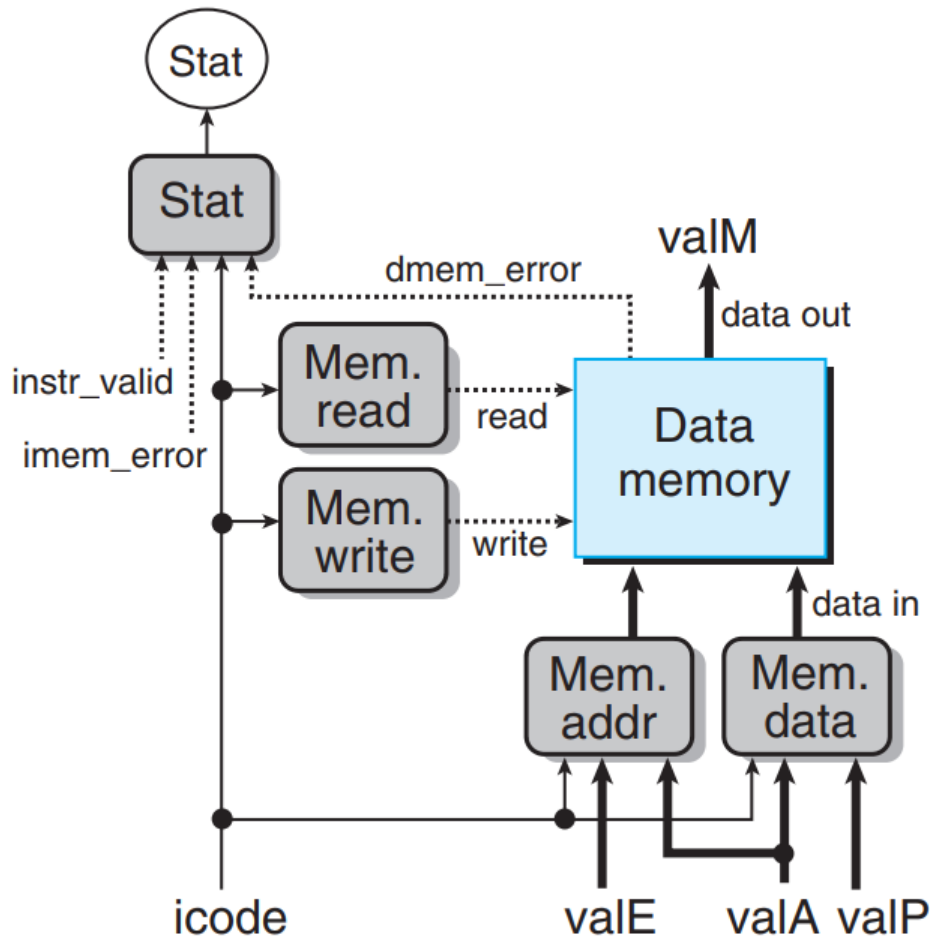
Stat: What is instruction status?

Mem. read: should word be read?

Mem. write: should word be written?

Mem. addr.: Select address

Mem. data.: Select data



```

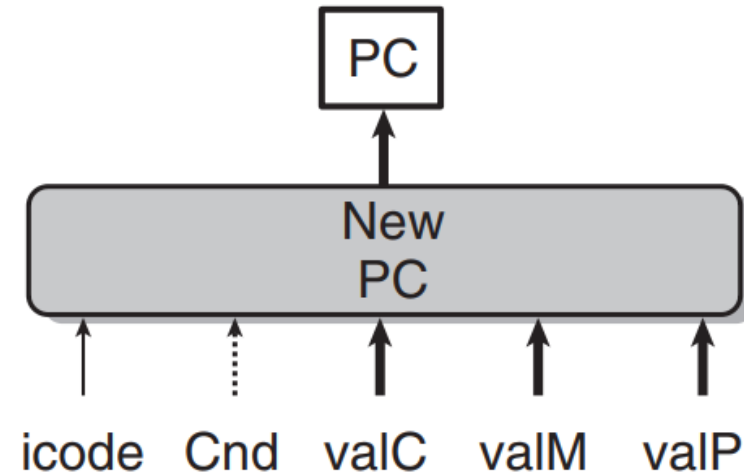
#Determine instruction status
int Stat = [
    imem_error || dmem_error : SADR;
    !instr_valid: SINS;
    icode == IHALT : SHLT;
    1 : SAOK;
];
  
```



Figure 4.31

SEQ PC update stage. The next value of the PC is selected from among the signals `valC`, `valM`, and `valP`, depending on the instruction code and the branch flag.

```
int new_pc = [  
    icode == ICALL : valC;  
    icode == IJXX && Cnd : valC;  
    icode == IRET : valM;  
    1 : valP;  
];
```



PART 04

SEQ timing

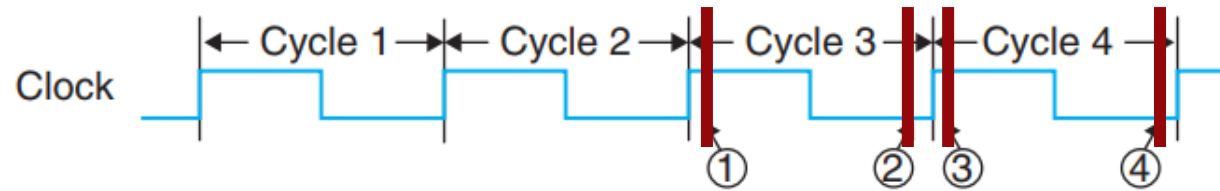
Instruction

Clock

Cycle



SEQ Operation

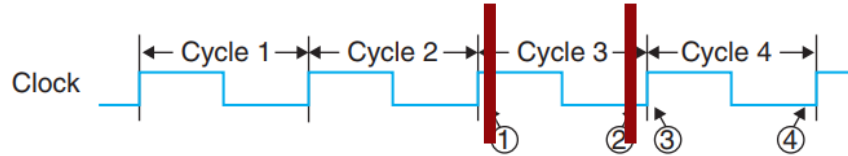


Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je dest	# Not taken
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

State of PC register, CC register, data memory, register file, updated only as clock rises

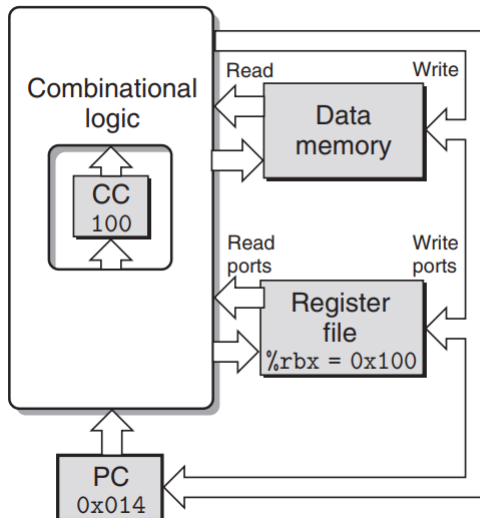
Combinational logic including ALU, control logic, memory reads (instruction memory, register file, data memory)

SEQ Operation #1 #2

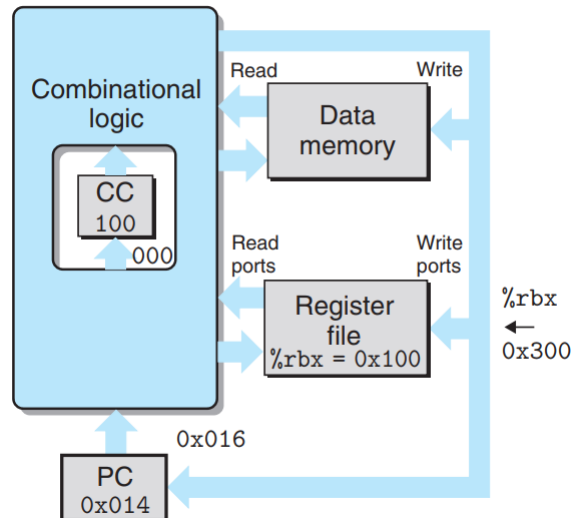


Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je dest	# Not taken
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

① Beginning of cycle 3



② End of cycle 3



State set according to second
irmovq instruction

#1 combinational logic starting to
react to state changes

#2 combinational logic generates
results for **addq** instruction

SEQ Operation #3 #4

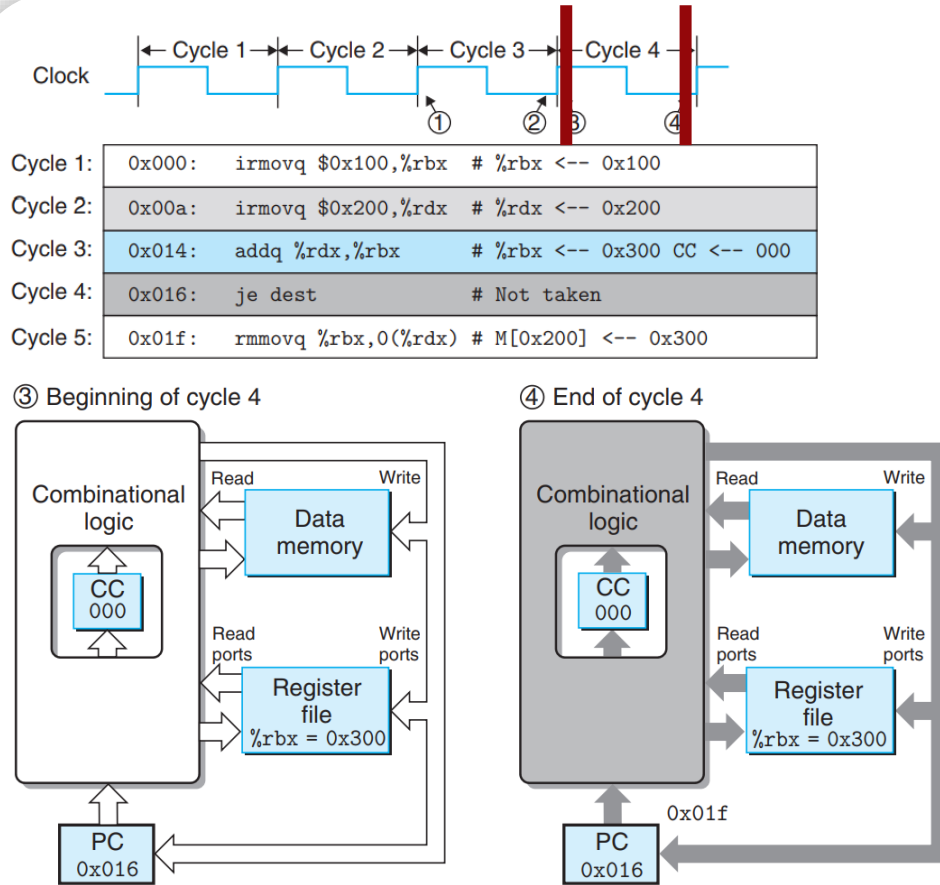


Figure 4.25 Tracing two cycles of execution by SEQ. Each cycle begins with the state elements (program counter, condition code register, register file, and data memory) set according to the previous instruction. Signals propagate through the combinational logic, creating new values for the state elements. These values are loaded into the state elements to start the next cycle.

State set according to second
addq instruction

#3 combinational logic starting to
react to state changes

#4 combinational logic generates
results for **je** instruction

By organizing the steps required to execute each of the different instructions into **a uniform flow**, we can implement the entire processor with **a small number of different hardware units** and with **a single clock** to control the sequencing of computations.

The control logic must then **route the signals** between these units and generate the proper control signals based on the instruction types and the branch conditions.

The clock must run **slowly enough** so that signals can propagate through all of the stages within a single cycle.

This style of implementation does not make very good use of our hardware units, since each unit is only active for a fraction of the total clock cycle.

We will see that we can achieve much better performance by introducing **pipelining**.

Thanks

Oct 22, 2025

